

Plonkify: R1CS-to-Plonk transpiler

Pengfei Zhu

zhupf321@gmail.com

Abstract

Rank-1 Constraint Systems (R1CS) and Plonk constraint systems are two commonly used circuit formats for zero-knowledge succinct non-interactive arguments of knowledge (zkSNARKs). We present Plonkify, a tool that converts a circuit in an R1CS arithmetization to Plonk, with support for both vanilla gates and custom gates. Our tool is able to convert an R1CS circuit (compiled from the Circom circuit description language) with 250,938 constraints to a vanilla Plonk circuit with 589,829 constraints, or a jellyfish turbo Plonk circuit with 370,086 constraints. This represents a $3.8\times$ and $2.0\times$ reduction in the number of constraints over the respective naïve conversions.

Further, we make several optimizations to the Circom compiler in order to minimize the number of non-zero elements in the generated R1CS circuits, and to facilitate conversion to Plonks. When recompiled with our optimized version of Circom, the aforementioned circuit sees a 49% reduction in the number of non-zero elements, with only a 0.4% increase in the number of constraints. The same circuit can now be represented in just 422,610 vanilla Plonk constraints, or 312,163 high-degree ones.

1. Introduction

In recent years, zkSNARKs (zero-knowledge succinct non-interactive arguments of knowledge) have seen various applications in blockchains and verifiable machine-learning. In blockchains, zkSNARKs are the basis of *cross-chain bridges*, where a proof for the correct evaluation of a ‘light client’ on one chain is posted on another [20]. This allows users on the chain to be ascertained of events on the other, enabling cross-chain transfers and other applications. For machine learning, there has been much academic interest (though not yet practical applications), with works such as zkLLM [19] focusing on the privacy of model parameters as well as the credibility of model evaluation.

Current zkSNARKs aim to prove the correct evaluation of a *circuit* defined in a certain format, also known as an *arithmetization*. There are two main types of arithmetizations used in zkSNARKs. One is the Rank-1 constraint

system (R1CS) [10, 4, 16], and the other is the Plonk constraint system (Plonk) [9, 13, 7]. R1CS allows for free addition gates, while Plonk circuits can be more easily adapted to high-degree constraints. In addition, *lookup arguments* [8, 7] have been developed for Plonk-based systems, which allow for efficient implementation of non-linear gates. We will review the two circuit formats in detail in the following sections.

While the two formats are similar (notably, the vanilla Plonk is a special case of R1CS), there are no tools to efficiently convert the formats from one to another. Pianist [13] allows for a crude conversion from R1CS to Plonk, but it has a large overhead - an R1CS from Polygon Hermez with 261,833 constraints is converted into a Plonk circuit with 2,544,486 constraints, a nearly $10\times$ increase.

In this technical report, we introduce Plonkify, a tool to convert R1CS into Plonk while minimizing the number of constraints. We hope that our tool will be of use for those with existing R1CS circuits and looking to adopt Plonk-based schemes, and we believe that it will also serve as a good reference when comparing the performance of R1CS-based and Plonk-based proof systems.

2. Preliminaries

The Rank-1 constraint system (R1CS) is defined as follows [4].

Definition 1 (R1CS). *Given coefficient matrices $A, B, C \in \mathbb{F}^{m \times n}$ and public input $\mathbf{v} \in \mathbb{F}^k$ ($k \leq n$), the R1CS instance defined by $A, B, C, \mathbf{v}$ is satisfied if there exists a witness $\mathbf{w} \in \mathbb{F}^n$ such that $\mathbf{w}[1 : k] = \mathbf{v}$, and $A\mathbf{w} \circ B\mathbf{w} = C\mathbf{w}$, where $\circ$ is elementwise multiplication.*

An R1CS can thus be seen as m constraints of the form

$$\langle \mathbf{q}_a, \mathbf{w} \rangle \cdot \langle \mathbf{q}_b, \mathbf{w} \rangle = \langle \mathbf{q}_c, \mathbf{w} \rangle$$

where the selector vectors $\mathbf{q}_a, \mathbf{q}_b, \mathbf{q}_c$ are rows of the coefficient matrices. By convention, the first element of $\mathbf{v}$ (and $\mathbf{w}$) is set to 1, such that constant terms can be encapsulated in the constraints.

It is worth noting that each element of the witness can be involved in several constraints. If we picture the constraints

as *gates* in a circuit and the witness as values of the *wires*, the correct connections between wires in different gates are ensured by selecting the same element in the witness.

In its most general form, the Plonk constraint system is defined as follows [7].

Definition 2 (Plonk). Let $Q \in \mathbb{F}^{m \times k}$ be the selector matrix and $W \in \mathbb{F}^{m \times n}$ be the witness matrix. (Q_i and W_i refer to the i -th row of the corresponding matrices.) Let $f(\mathbf{q}, \mathbf{w})$ be the common gate equation where $\mathbf{q} \in \mathbb{F}^k$, $\mathbf{w} \in \mathbb{F}^n$ are rows of Q and W , respectively. Let σ be a prespecified permutation on the positions of W . The Plonk instance is satisfied if all of the following hold:

- *Gate identity:* $\forall i, f(Q_i, W_i) = 0$.
- *Wiring identity:* $\forall i, j, W_{\sigma(i,j)} = W_{ij}$.
- *Public input consistency:* Given a public input $\mathbf{v} \in \mathbb{F}^p$, $W_1[1 : p] = \mathbf{v}$.

In Plonk, each constraint has its own row of witnesses; if pictured as a circuit, each gate has its own *loose* input and output wires. The wiring identity is introduced to connect these loose wires. That is to say, certain positions of the witness matrix are required to be equal to each other (e.g. the output of gate 1 might be connected to the left input of gate 3). Plonk handles this with a prespecified permutation σ . This permutation is carefully selected such that it runs a cycle through each equivalence class. Therefore, all the equivalence relations are reduced to a claim that $W_{\sigma(i,j)} = W_{ij}$.

The vanilla Plonk [9] uses gate equations of the form

$$q_L w_L + q_R w_R + q_O w_O + q_M w_L w_R + q_C = 0,$$

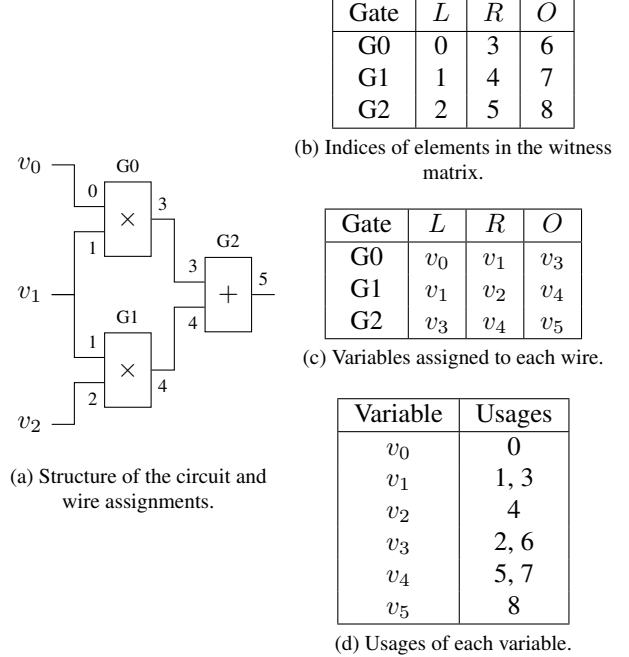
where q are the selectors and w are the witnesses. An addition gate can be implemented by setting $q_L = q_R = 1, q_O = -1, q_M = q_C = 0$, while a multiplication gate may be $q_O = -1, q_M = 1, q_L = q_R = q_C = 0$.

3. Our techniques

We detail our techniques in this section. We evaluate our tool on a Circom circuit from Polygon-Hermez [11] with 229,847 R1CS constraints. As a baseline, Pianist’s [13] open-source implementation converts it to 2,215,685 Plonk constraints.

3.1. Theoretical discussion

Our task can be formulated as an optimization problem. It is likely NP-hard to find the optimal solution, and even if there exists a polynomial-time algorithm, the optimization space is so large that it will likely be impractical. We referred to existing problems such as the Steiner tree problem and the survivable network design problem, but none of



$$\sigma = (0)(1\ 3)(4)(2\ 6)(5\ 7)(8)$$

Figure 1: Example of translating the circuit structure and variable assignments into the Plonk instance’s permutation.

them appears applicable to our case. Therefore, our converters are all based on intuition and heuristics, aimed at striking a balance between optimality (number of constraints) and concrete run-time.

3.2. Circuit structure: permutations vs variables

As mentioned in the preceding sections, a Plonkish circuit has a witness matrix instead of a witness vector, and it relies on a permutation to ensure the correct connections of wires between the gates/constraints. However, during circuit generation, we found it helpful to first generate an R1CS-style circuit, in which wires on each gate are assigned variable indices. If two wires are assigned the same indices, they are ‘connected’ and are required to hold the same value by the circuit’s structure. In the end, the permutation is constructed by checking which wires each variable has been assigned to, and running a cycle through the wires that use each variable. For an illustration, check out Figure 1, where we demonstrate this process for a simple circuit.

3.3. A naïve convertor

Recall that an R1CS contains constraints of the form

$$\langle \mathbf{q}_a, \mathbf{w} \rangle \cdot \langle \mathbf{q}_b, \mathbf{w} \rangle = \langle \mathbf{q}_c, \mathbf{w} \rangle.$$

Since the vanilla Plonk is degree-2, it seems reasonable to compute $L_a = \langle \mathbf{q}_a, \mathbf{w} \rangle$, $L_b = \langle \mathbf{q}_b, \mathbf{w} \rangle$ and $L_c = \langle \mathbf{q}_c, \mathbf{w} \rangle$,

store these values on auxiliary wires, and then apply the Plonk constraint

$$L_a L_b - L_c = 0.$$

A naïve computation of L_a requires $N_a - 1$ Plonk constraints if $\mathbf{q}_a$ has N_a non-zero terms, and the same is true for L_b and L_c . Therefore, each RICS constraint is converted into $N_a + N_b + N_c - 2$ constraints (add 1 for the final multiplication). This is exactly what Pianist does, as shown in Algorithm 1.

Algorithm 1 Pianist’s naïve convertor

Require: RICS instance $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}_{m \times n}, \mathbf{w} \in \mathbb{F}^n$
Ensure: Vanilla Plonk instance $\mathbf{Q} \in \mathbb{F}_{p \times 5}, \sigma, \mathbf{W} \in \mathbb{F}_{p \times 3}$
procedure SUMLC($\mathbf{q}$) $\triangleright$ Process a linear combination
 $(j_0, q_0) \leftarrow \mathbf{q}_1$ $\triangleright$ variable j and coefficient q
 for each non-zero entry $(j, q) = \mathbf{q}_2, \mathbf{q}_3, \dots$ **do**
 Add a variable j' , and assign $v_{j'} \leftarrow q_0 v_{j_0} + q v_j$
 Add a Plonk constraint $q_0 v_{j_0} + q v_j = v_{j'}$
 $(j_0, q_0) \leftarrow (j', 1)$
 end for
 return (j_0, q_0)
end procedure
for each row $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{F}^m$ **in** $\mathbf{A}, \mathbf{B}, \mathbf{C}$ **do**
 $(j_a, q_a) \leftarrow \text{SUMLC}(\mathbf{a})$
 $(j_b, q_b) \leftarrow \text{SUMLC}(\mathbf{b})$
 $(j_c, q_c) \leftarrow \text{SUMLC}(\mathbf{c})$
 Add a Plonk constraint $q_a q_b v_{j_a} v_{j_b} = q_c v_{j_c}$
end for
 Create σ from the variable assignments

3.4. The slightly less naïve convertor

Pianist’s convertor does not take into consideration the fact that the first variable in each RICS is fixed to 1 by convention, so $\mathbf{w}_1$ can be considered a constant term. Suppose that the linear combinations are $L_a = X_a + C_a$, $L_b = X_b + C_b$, $L_c = X_c + C_c$, where C are the constant terms, we observe that the equation

$$L_a L_b - L_c = 0$$

can be expanded to

$$X_a X_b + C_b X_a + C_a X_b - X_c + (C_a C_b - C_c) = 0$$

which can still be represented with a single Plonk constraint. It is therefore not necessary to generate constraints to add up X_a and C_a , etc.

We observe that, since X_c is linear, it is not necessary to reduce X_c to a single term; we only need to ensure that it introduces at most 1 new variable. Here, ‘variables’ do

not include constants. For example, the following equation would be acceptable:

$$(q_1 w_1 + q_2)(q_3 w_2 + q_4) = q_5 w_1 + q_6 w_2 + q_7 w_3 + q_8.$$

(Note how w_1 appears in both X_a and X_c .)

With these observations, we devise our version of the naïve convertor - it generate constraints to compute X_a, X_b and X_c , except that for X_c it stops once only one new variable remains. No attempt is made to eliminate repetitive addition gates. This simple construction is extremely fast and yet still offers a notable advantage over the very similar convertor in Pianist. It converts the circuit to 1,845,578 Plonk constraints. We sketch the algorithm in Algorithm 2.

3.5. Segment-tree-inspired optimized convertor

The naïve convertor is obviously less than ideal - since the RICS contains many similar linear combinations, there is a lot of repeated computation to build each linear combination from scratch. It is natural to build upon the naïve construction by adding memoization. Recall that each Plonk constraint adds up two variables to produce a new intermediate variable. If the same two variables are added again with the same ratio of coefficients, we can simply reuse the existing intermediate variable (possibly with a different coefficient).

This immediately brings the total number of constraints down to about 1.2 million. However, there is still a lot of potential. For an example, suppose that there are two linear combinations

$$w_1 + w_2 + w_3 + w_4 + \dots + w_{10},$$

$$w_2 + w_3 + \dots + w_{10}.$$

If the terms are added up in variable order, no intermediate values produced in either computation can be used for the other, because all of them will be different. (For the first, the intermediate values will be $w_1 + w_2, w_1 + w_2 + w_3$, etc, while for the second they will be $w_2 + w_3, w_2 + w_3 + w_4$, etc.) Of course, in this example, if the variables are summed in reverse order, the problem will be resolved. However, it is impossible to know what the ‘optimal’ order is. Moreover, specifying a single order in which variables are added to the running sum is not enough; consider the following example of three linear combinations

$$w_1 + w_2 + w_3 + w_{10} + w_{11} + w_{12},$$

$$w_1 + w_2 + w_3,$$

$$w_{10} + w_{11} + w_{12}.$$

the only way that allows the first linear combination to reuse results from the others would be to sum $w_1 + w_2 + w_3$ and $w_{10} + w_{11} + w_{12}$ separately and then add the two sums up.

Algorithm 2 Our ‘slightly less naïve’ convertor**Require:** RICS instance $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}_{m \times n}$, $\mathbf{w} \in \mathbb{F}^n$ **Ensure:** Vanilla Plonk instance $\mathbf{Q} \in \mathbb{F}_{p \times 5}$, σ , $\mathbf{W} \in \mathbb{F}_{p \times 3}$ **procedure** SUMLC($\mathbf{q}$) $\triangleright$ Process a linear combination $c_0 \leftarrow$ Sum of constant terms in $\mathbf{q}$ Remove constant terms in $\mathbf{q}$ **if** $\mathbf{q}$ is now empty **then****return** $(0, 0, c_0)$ **end if** $(j_0, q_0) \leftarrow \mathbf{q}_1$ $\triangleright$ variable j and coefficient q **for** each non-zero entry $(j, q) = \mathbf{q}_2, \mathbf{q}_3, \dots$ **do**Add a variable j' , and assign $v_{j'} \leftarrow q_0 v_{j_0} + q v_j$ Add a Plonk constraint $q_0 v_{j_0} + q v_j = v_{j'}$ $(j_0, q_0) \leftarrow (j', 1)$ **end for****return** (j_0, q_0, c_0) **end procedure****procedure** MULCONSTRAINT($j_a, q_a, c_a, j_b, q_b, c_b, \mathbf{c}$) $(q_L, q_R, q_O, q_M, q_C) \leftarrow (c_b q_a, c_a q_b, 0, q_a q_b, c_a c_b)$ $j_c \leftarrow 0$ **for** each non-zero entry (j, q) in $\mathbf{c}$ **do****if** $j = 0$ **then** $q_C \leftarrow q_C - q$ **else if** $j = j_a$ **then** $q_L \leftarrow q_L - q$ **else if** $j = j_b$ **then** $q_R \leftarrow q_R - q$ **else** $j_c \leftarrow j, q_O \leftarrow q_O - q$ **end if****end for**

Add a Plonk constraint with selectors

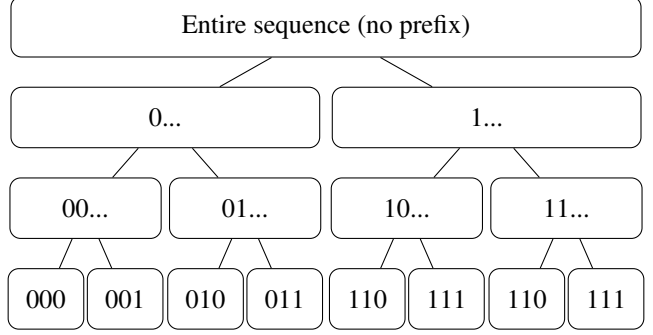
 $(q_L, q_R, q_O, q_M, q_C)$ and variables (j_a, j_b, j_c) **end procedure****for** each row $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{F}^m$ in $\mathbf{A}, \mathbf{B}, \mathbf{C}$ **do** $(j_a, q_a, c_a) \leftarrow \text{SUMLC}(\mathbf{a})$ $(j_b, q_b, c_b) \leftarrow \text{SUMLC}(\mathbf{b})$ Split $\mathbf{c}$ into two parts $\mathbf{c}_1, \mathbf{c}_2$, such that $\mathbf{c}_1$ contains all j_a and j_b terms $(j_c, q_c, c_c) \leftarrow \text{SUMLC}(\mathbf{c}_2)$ Append (j_c, q_c) and $(0, c_c)$ to $\mathbf{c}_1$ Call MULCONSTRAINT($j_a, q_a, c_a, j_b, q_b, c_b, \mathbf{c}_1$)**end for**Create σ from the variable assignments

Figure 2: Structure of a segment tree

on layer k each represent the sum over a segment with a common k -bit prefix. See Figure 2 for an illustration.

We will represent each term $q \cdot w_j$ in a linear combination with a pair (j, q) , where j is the index of the witness variable and q is the coefficient. A linear combination is represented as a set of such pairs. Our algorithm starts on the bottom-most layer, and first adds up all terms $(j_1, q_1), (j_2, q_2)$ ($j_1 < j_2$) such that the binary representations of j_1 and j_2 differ only in the last bit (i.e. they are siblings on the last layer of the segment tree). The sum is stored in the position for j_1 ; then we add up all the terms with indices that differ only in the last two bits, and so on, until the linear combination is reduced to a single term. We provide a more formal description of the algorithm in Algorithm 3.

To see how this technique can help better reuse the intermediate values, we will study a concrete example. Suppose that there are 16 variables labelled 0 to 15, and there are two linear combinations

$$w_0 + w_1 + w_4 + w_5 + w_7 + w_{13} + w_{14} + w_{15},$$

$$w_2 + w_3 + w_4 + w_5 + w_{11} + w_{13} + w_{14} + w_{15}.$$

We show both linear combinations in segment-tree-style diagrams in Figure 3.

We can immediately see that we are able to reuse $w_4 + w_5$ and $w_{13} + w_{14} + w_{15}$, because the large linear combinations have been broken into smaller parts. While this is an ideal example, even in cases where the variables do not match up perfectly, plenty of segments can still be reused. (As in a segment tree, sometimes it is necessary to combine several small segments to form a segment with a length that is not a power of two.)

This convertor is fast as it still only processes each RICS constraint once, but it does require some extra memory for memoization. It is able to convert the Polygon circuit to just 934,149 Plonk constraints.

3.6. “Greedy brute-force” convertor

For an even higher optimization level, consider the following greedy heuristic. Since each Plonk constraint com-

We take inspiration from the structure of segment trees. In a segment tree, each node on layer k represents the sum of two nodes on layer $k+1$. Specifically, the first and second nodes in layer $k+1$ are added up to form the first node on layer k , and the third and fourth nodes are added up for the second, etc. If we consider the binary representations of the indices of the leaf nodes, we will see that the 2^k nodes

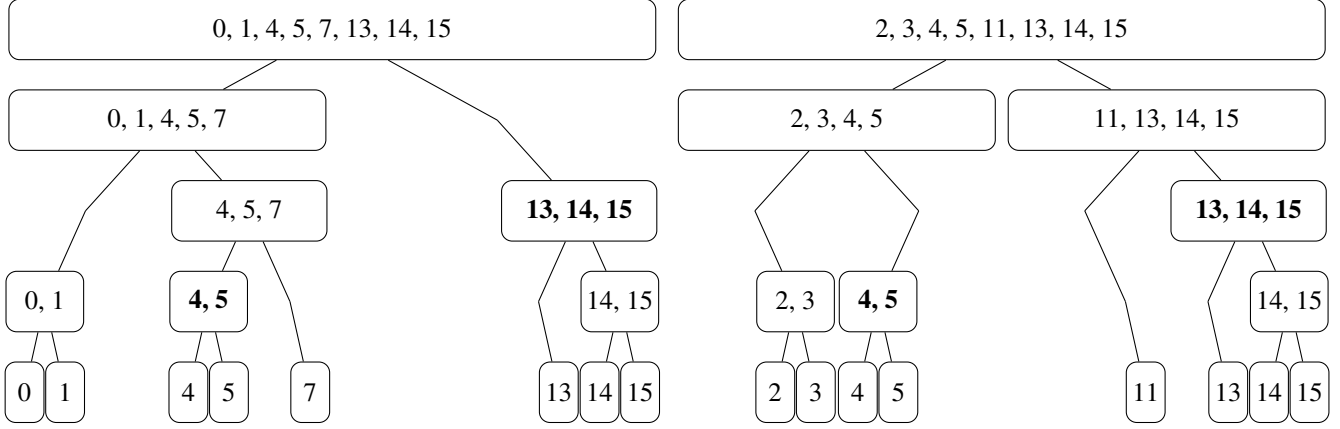


Figure 3: Illustration of the two linear combinations (emphasis added where segments can be reused)

Algorithm 3 Key parts of our optimized convertor

```

 $H \leftarrow$  empty hash table
procedure ADDITION( $j_a, q_a, j_b, q_b$ )
   $q \leftarrow q_b/q_a$ 
  if  $\exists j', ((j_a, j_b, q), j') \in H$  then
    return ( $j', q_a$ )
  else
    Add  $v_{j'} \leftarrow v_{j_a} + qv_{j_b}$ 
    Add a constraint  $v_{j_a} + qv_{j_b} = v_{j'}$ 
    Add  $((j_a, j_b, q), j')$  to  $H$ 
    return ( $j', q_a$ )
  end if
end procedure
procedure SUMLC( $\mathbf{q}$ )  $\triangleright$  Process a linear combination
   $c_0 \leftarrow$  Sum of constant terms in  $\mathbf{q}$ 
  Remove constant terms in  $\mathbf{q}$ 
  if  $\mathbf{q}$  is now empty then
    return (0, 0,  $c_0$ )
  end if
   $L \leftarrow \{(j, q, j) \mid (j, q) \in \mathbf{q}\}$ 
   $k \leftarrow 1$ 
  while  $|L| > 1$  do
    for adjacent  $(j_1, q_1, k_1), (j_2, q_2, k_2) \in L$  do
      if  $k_1$  and  $k_2$  differs only in the last  $k$  bits then
         $(j', q') \leftarrow \text{ADDITION}(j_1, q_1, j_2, q_2)$ 
        Remove  $(j_2, q_2, k_2)$  from  $L$ 
        Replace  $(j_1, q_1, k_1)$  with  $(j', q', k_1)$  in  $L$ 
      end if
    end for
     $k \leftarrow k + 1$ 
  end while
   $(j_0, q_0, k_0) \leftarrow L_1$ 
  return ( $j_0, q_0, c_0$ )
end procedure

```

binest two variables at a given ratio into a new variable, each time we are to add a new constraint, we should select a pair of variables and a ratio such that the most number of non-zero terms can be eliminated across all pending linear combinations.

In our algorithm (Algorithm 4), we first find all pairs of variables in all the linear combinations. We record the places of occurrences of each pair (variable 1, variable 2, and ratio), and put those pairs with more than 1 occurrence in a priority queue. Each round, we take the pair with the most occurrences and create a constraint to replace all the occurrences of that pair with a new intermediate variable. Then, it is necessary to update the data structures - first, update the linear combinations that use the pair; then, for each updated linear combination, remove the records of pairs that no longer occur (not just the pair replaced, but also pairs involving one removed variable and another variable in the linear combination); and finally, add new occurrences of pairs (the new intermediate variable along with another variable in the linear combination). The number of occurrences recorded in the priority queue are updated accordingly.

This convertor can be said to be somewhat ‘brute-force’, and it has high time and memory costs. However, it is still practical for reasonably-sized circuits such as the one from Polygon-Hermez, which is now able to be represented in just 855,296 Plonk constraints.

3.7. Custom gates convertor

One of the strengths of Plonkish proof systems is the ability to utilize high-degree custom gates. The conversion of an existing R1CS to a high-degree Plonk has distinctive difficulties. Since an R1CS encodes only degree-2 information, it is necessary to recover the high-degree relations scattered across multiple R1CS constraints. For instance, to

Algorithm 4 Key parts of our Greedy-BF convertor

procedure PROCESSLCs(**L**) $\triangleright$ All LCs from **A**, **B**, **C**
Remove constant terms from LCs in **L** into **c**
 $H \leftarrow$ empty hash table of lists $\triangleright H$ is keyed by
 (j_a, j_b, q) such that $j_a < j_b$. If $j_a > j_b$, for convenience
we write $H[j_a, j_b, q] = H[j_b, j_a, 1/q]$
for $i \in [1, |\mathbf{L}|]$ **do**
 $\mathbf{l} \leftarrow \mathbf{L}_i$
 for $a, b \in [1, |\mathbf{l}|], a < b$ **do**
 $(j_a, q_a), (j_b, q_b) \leftarrow \mathbf{l}_a, \mathbf{l}_b$
 $q \leftarrow q_b/q_a$
 Insert i into the list at $H[j_a, j_b, q]$
 end for
end for
 $P \leftarrow$ empty priority queue
for $((j_a, j_b, q), I) \in H$ **do** $\triangleright I$ is a list of indices
 Insert $(|I|, (j_a, j_b, q))$ into P
end for
while P is not empty **do**
 $(|I|, (j_a, j_b, q)) \leftarrow \text{POP}(P)$
 Add $v_{j'} = v_{j_a} + qv_{j_b}$ and constraint
 for $i \in H[j_a, j_b, q]$ **do**
 $\mathbf{l} \leftarrow \mathbf{L}_i$
 $q_a, q_b \leftarrow$ coefficient of j_a, j_b in $\mathbf{l}$
 Substitute v_{j_a}, v_{j_b} in $\mathbf{l}$ with $q_a v_{j'}$
 for $(j, q) \in \mathbf{l}, j \neq j'$ **do**
 Remove i from $H[j_a, j, q/q_a]$
 Remove i from $H[j_b, j, q/q_b]$
 Add i to $H[j, j', q_a/q]$
 end for
 end for
 Update P with the changes to $|I|$ in H
end while
end procedure

represent $y = x^5$, an RICS might use 3 constraints:

$$\begin{aligned}x_2 &= x \cdot x, \\x_4 &= x_2 \cdot x_2, \\y &= x_4 \cdot x.\end{aligned}$$

It is necessary to eliminate the intermediate variables and constraints to realize that the system in fact encodes a simple degree-5 relation.

We tackle this problem by first constructing an *expanded circuit* which includes high-degree equations. Since each RICS instance represents the satisfaction of a circuit, all the intermediate wires should be able to be computed from the inputs. Therefore, it seems to be a reasonable intuition that each constraint in the RICS either introduces a new intermediate wire (by a constraint that relates the new wire to previous ones) or introduces a constraint on existing wires.

In the latter case, these ‘existing wires’ may include previous intermediate wires, which can eventually be substituted and reduced to an expression with the original inputs. Therefore, the RICS can be reduced to a few very complex high-degree constraints on the input variables. Intuitively, this would recover the high-degree relations inherent in the RICS. However, in practice, doing so will cause both the number of terms in the constraints and the degree of the constraints to grow exponentially, and the number of Plonkish gates (even custom ones) required to represent each constraint will skyrocket.

Therefore, we designed a few rules to limit the expansion. We use a parameter that limits the maximum degree and number of terms of each expanded constraint. When an intermediate variable can be represented by another variable at no additional cost (e.g. $x_1 = kx_3$), that variable will always be eliminated; when the replacement is high-degree but is still a monomial (e.g. $x_1 = kx_2x_3^2$), the replacement is attempted for all dependents of x_1 , and confirmed if no dependent exceeds the prespecified limits on degree and number of terms after substitution. When the expression has more than one term, e.g. $x_1 = kx_2 + zx_3^2$, replacement is only attempted if there is only one use of x_1 , and after replacement the expanded constraint satisfies the prespecified limits. We also have special considerations for outlining long linear combinations as new variables. There are lots of these in an RICS, and they can result in high costs if not outlined (e.g. 50 terms times 50 terms expands to 2,500 terms).

The next step is to convert the expanded circuit to a Plonkish instance. We again employ a greedy heuristic. The full algorithm is quite complicated, and we summarize it as follows. We process the expanded constraints one by one. For each constraint, in each round we start with the term with the highest degree. We try to fit that term into the gate equation to reduce its cost (in this instance, degree) as much as possible; and then, we try to add in other terms along the way to reduce the total estimated cost of the expanded constraint. This is done with DFS, but since there are plenty of linear terms in the expanded constraint, linear terms are handled separately to reduce the time costs of the search. This process of reducing the highest-degree term continues until only linear terms remain. Finally, linear terms are handled with a similar heuristic as the segment-tree-inspired convertor for vanilla gates. (Deduplication through memoization is in place for all the constraints added.) We also have a brute-force preprocessing option to try and outline LCs; this is quite slow and saves only a few constraints, but we nevertheless include it in the evaluation because it represents the best custom-gate convertor we offer.

We will study two examples to illustrate what we mean by ‘add in other terms along the way’. Assuming that we use the jellyfish turbo gate:

- $x^5 + x^4 + x^2 + x - 15 = 0$ can be represented with one constraint. (Set $w_1 = w_2 = w_3 = w_4 = x$.)
- $x^7 + x^2y^2 - 10 = 0$ needs constraints: $x^5 + y^2 = z$, $x^2z - 10 = 0$.

For a baseline, we also developed a linear-only converter, where the custom gate is only ever used for additions to build LCs and degree-2 multiplication, i.e. similar to vanilla gates. The custom gate still has an advantage because it has higher fan-in and as such allows for more terms to be summed up with one gate. We report the results both with and without our segment-tree tricks ('Naïve').

4. Nonzero-oriented Circom optimization

The current behavior of Circom is to optimize for constraint count. This is appropriate for some proof systems like Groth16, but the efficiency of many R1CS-based proof systems depend on the number of *nonzero elements* rather than the number of constraints. The same is true for Plonkify: fewer nonzero elements will lead to fewer constraints in the converted Plonk.

It is easy to see that the two optimization goals are sometimes contradictory. To minimize constraint count, one should repeat (inline) linear combinations as much as possible; an example might be

$$\begin{cases} (3x_1 + 5x_2 + 6x_3)(4x_4 + 5x_6 + 6x_7) = 7x_8 + 8x_9 + 9x_{10}, \\ (3x_1 + 5x_2 + 6x_3)(7x_8 + 8x_9 + 9x_{10}) = 4x_4 + 5x_6 + 6x_7, \\ (4x_4 + 5x_6 + 6x_7)(7x_8 + 8x_9 + 9x_{10}) = 3x_1 + 5x_2 + 6x_3. \end{cases}$$

To minimize the number of nonzero elements, one should instead outline the linear combinations as auxiliary variables. The same R1CS might now be rewritten as

$$\begin{cases} 3x_1 + 5x_2 + 6x_3 = z_1, \\ 4x_4 + 5x_6 + 6x_7 = z_2, \\ 7x_8 + 8x_9 + 9x_{10} = z_3, \\ z_1z_2 = z_3, \\ z_1z_3 = z_2, \\ z_2z_3 = z_1. \end{cases}$$

We extended Circom and added an option to optimize for the number of nonzero elements instead. With this option enabled, the number of nonzero elements sees dramatic decreases, with only a slight increase in constraint count. This is because we avoid some of the worst substitutions. This nonzero-oriented Circom is suitable for nonzero-based proof systems, and is also useful in conjunction with Plonkify to reduce the sizes of Plonk circuits produced.

5. Evaluation

The source code of Plonkify is available at <https://github.com/zhaowenlan1779/plonkify>. The

source code of the optimized Circom is available at <https://github.com/zhaowenlan1779/circom/tree/plonkish-opt>.

We conducted experiments on one Aliyun server with a 16-core Intel Xeon Platinum 8575C and 256GB of memory. We used one circuit from Polygon Hermez [14] (Circom 1) and four circuits from circom-ecdsa [1] (Circom 2). Our Circom optimization is based on Circom 2, so the circuit from Polygon Hermez is not included for experiments with the optimized Circom compiler. Basic information about the circuits are shown in Table 1. We list key results in Tables 2 to 9.

References

- [1] 0xPARC. circom-ecdsa: Big integer arithmetic and secp256k1 ECC operations in circom. <https://github.com/0xPARC/circom-ecdsa>. Accessed: 2025/5/18. 7
- [2] Miguel Ambrona, Anne-Laure Schmitt, Raphael R. Toledo, and Danny Willems. New optimization techniques for PlonK's arithmetization. Cryptology ePrint Archive, Paper 2022/462, 2022. 9
- [3] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Paper 2018/046, 2018. 8
- [4] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. Aurora: Transparent succinct arguments for R1CS. In *Advances in Cryptology - EUROCRYPT 2019: 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19–23, 2019, Proceedings, Part I*, pages 103–128, Berlin, Heidelberg, 2019. Springer-Verlag. 1, 8, 9
- [5] Sean Bowe, Jack Grigg, and Daira Hopwood. Recursive proof composition without a trusted setup. Cryptology ePrint Archive, Paper 2019/1021, 2019. 8
- [6] Stefanos Chaliasos, Itamar Reif, Adria Torralba-Agell, Jens Ernstberger, Assimakis Kattis, and Benjamin Livshits. Analyzing and benchmarking ZK-rollups. Cryptology ePrint Archive, Paper 2024/889, 2024. 8
- [7] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. HyperPlonk: Plonk with linear-time prover and high-degree custom gates. Cryptology ePrint Archive, Paper 2022/1355, 2022. <https://eprint.iacr.org/2022/1355>. 1, 2, 9, 10
- [8] Ariel Gabizon and Zachary J. Williamson. plookup: A simplified polynomial protocol for lookup tables. Cryptology ePrint Archive, Paper 2020/315, 2020. <https://eprint.iacr.org/2020/315>. 1, 9
- [9] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Paper 2019/953, 2019. <https://eprint.iacr.org/2019/953>. 1, 2, 10

Circuit	Constraints	Nonzero elements	Nonzero density
polygon	229,847	2,376,228	10.34
pubkeygen	95,444	1,653,507	17.32
eth_addr	247,380	2,389,248	9.66
groupsig	250,938	2,404,813	9.58
verify	1,508,904	9,836,175	6.52

Table 1: Circuits used in the experiments

Circuit	Baseline	Naïve	Optimized	Greedy-BF
polygon	2,215,685	1,845,578	934,149	855,296
pubkeygen	1,646,748	1,457,018	423,853	417,601
eth_addr	2,231,065	1,646,410	605,589	584,913
groupsig	2,243,329	1,651,590	610,837	589,829
verify	9,753,692	6,798,931	3,779,308	3,707,511

Table 2: Evaluation results with vanilla gates: number of Plonk constraints

- [10] Jens Groth. On the size of pairing-based non-interactive arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology – EUROCRYPT 2016*, pages 305–326, Berlin, Heidelberg, 2016. Springer Berlin Heidelberg. [1, 8, 9](#)
- [11] Polygon Lab. zkProver - Polygon knowledge layer. <https://docs.polygon.technology/zkEVM/architecture/zkprover>. Accessed: 2025/2/14. [2, 8](#)
- [12] Chongrong Li, Pengfei Zhu, Yun Li, Cheng Hong, Wenjie Qu, and Jiaheng Zhang. HyperPianist: Pianist with linear-time prover and logarithmic communication cost. Cryptology ePrint Archive, Paper 2024/1273, 2024. [10](#)
- [13] Tianyi Liu, Tiancheng Xie, Jiaheng Zhang, Dawn Song, and Yupeng Zhang. Pianist: Scalable zkRollups via fully distributed zero-knowledge proofs. Cryptology ePrint Archive, Paper 2023/1271, 2023. <https://eprint.iacr.org/2023/1271>. [1, 2, 10](#)
- [14] Hermez Network. Hermez network circuits. <https://github.com/hermeznetwork/circuits>. Accessed: 2025/2/14. [7, 8](#)
- [15] =nil; Foundation. =nil; zkLLVM circuit compiler. <https://nil.foundation/blog/post/circuit-compiler>. Accessed: 2025/2/14. [10](#)
- [16] Srinath Setty. Spartan: Efficient and general-purpose zk-SNARKs without trusted setup. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020*, pages 704–737, Cham, 2020. Springer International Publishing. [1, 9](#)
- [17] Srinath Setty, Justin Thaler, and Riad Wahby. Unlocking the lookup singularity with Lasso. Cryptology ePrint Archive, Paper 2023/1216, 2023. <https://eprint.iacr.org/2023/1216>. [9](#)
- [18] Marco Stronati, Denis Firsov, Antonio Locascio, and Benjamin Livshits. Clap: a semantic-preserving optimizing eDSL for Plonkish proof systems, 2024. [10](#)
- [19] Haochen Sun, Jason Li, and Hongyang Zhang. zkLLM: Zero knowledge proofs for large language models, 2024. [1](#)

- [20] Tiancheng Xie, Jiaheng Zhang, Zerui Cheng, Fan Zhang, Yupeng Zhang, Yongzheng Jia, Dan Boneh, and Dawn Song. zkBridge: Trustless cross-chain bridges made practical, 2022. [1](#)

A. Literature review

A.1. Current zkSNARK applications

There have been various industrial deployments of zk-SNARKs (mostly zkRollups [6]). Several zkRollups are based on zkSTARKs [3].¹ These systems use a somewhat different computational model, where the *program* is specified in terms of permitted state transitions, and the goal is to prove that the initial state can be legally transformed to the final state within T steps. It has been shown [4] that the performance of the original construction is subpar compared to circuit-based systems that target R1CS or Plonk. zkSTARKs are adopted in Starknet and RISC0 Zeth, among others [6].

R1CS-based proof systems (most notably Groth16 [10]) have been adopted in several zkSNARK applications as well. The circuits of the original Polygon Hermez network are written in Circom [14]. In the latest version of Polygon Hermez [11], the circuits are instead generated with various tools, but the end result is again a Circom component. The Circom code is then compiled into an R1CS and proved with Groth16 [10]. The original version of Zcash (<https://z.cash/>) is based on the Halo proof system [5], which targets R1CS as well.

Several recent applications have elected to use Plonk instead, including Scroll and Aztec [6]. Scroll is built upon Halo2, which is based on the Plonk arithmetization; Zcash has also moved to Halo2. Aztec has developed its own

¹In principle, zkSTARK may refer to any zkSNARK that is transparent setup. In practice it often refers to the specific construction in [3].

polygon	Baseline	Naïve	Optimized	Greedy-BF
Constraints	2,215,685	1,845,578	934,149	855,296
Time	N/A	1.85s	5.50s	198s
Memory	N/A	1024MB	724MB	28.09GB

verify	Baseline	Naïve	Optimized	Greedy-BF
Constraints	9,753,692	6,798,931	3,779,308	3,707,511
Time	N/A	3.03s	13.7s	1011s
Memory	N/A	3.72GB	2.52GB	85.74GB

Table 3: Resource consumption of different convertors with vanilla gates

Circuit	Naïve	Linear-only	Custom	Custom*
polygon	817,505	497,156	432,730	429,166
pubkeygen	553,112	206,125	230,745	208,362
eth_addr	719,982	373,038	388,684	366,483
groupsig	724,822	376,652	392,477	370,086
verify	3,288,407	2,277,040	2,504,655	2,301,856

Table 4: Evaluation results with jellyfish turbo gates: number of Plonk constraints (asterisk indicates convertor with preprocessing outline LCs)

framework, Barretenberg, to generate Plonk circuits. This further underlines the lack of standardization in the compilation of Plonk circuits.

A.2. R1CS and Plonk arithmetizations

We discuss the strengths of the R1CS and Plonk arithmetizations.

Linear combinations (such as addition) are free in R1CS in the sense that no constraints are required for them. Instead, they can be expanded and expressed with the selector vectors. However, they are also not truly free if the size of an R1CS instance is measured not by the number of rows but by the number of nonzero entries in the coefficient matrices [4]. More nonzero entries translate to more computation in many R1CS proof systems [4, 16], with the notable exception of Groth16 based systems [10].

Recall that the vanilla Plonk uses gate equations of the form

$$q_L w_L + q_R w_R + q_O w_O + q_M w_L w_R + q_C = 0.$$

It is obvious that this is a special case of an R1CS constraint. Therefore, the vanilla Plonk is strictly less expressive than R1CS and requires more constraints for the same circuit. However, high-degree gate equations can be utilized to improve the efficiency of the proof system. Some techniques for designing custom gates are discussed in [2]. An example

of a high-degree gate mentioned in [7] is

$$\begin{aligned} & q_1 w_1 + q_2 w_2 + q_3 w_3 + q_4 w_4 + q_{M1} w_1 w_2 + q_{M2} w_3 w_4 \\ & + q_{H1} w_1^5 + q_{H2} w_2^5 + q_{H3} w_3^5 + q_{H4} w_4^5 \\ & + q_E w_1 w_2 w_3 w_4 + q_O w_5 + q_C = 0. \end{aligned}$$

One single constraint in this form can replace up to 23 R1CS constraints if the original circuit is representing a high-degree relation.

In addition, the structure of the Plonk arithmetization allows it to be easily enhanced with *lookup gates* from lookup arguments.

A.3. Lookup arguments

The basic R1CS and Plonk arithmetizations are designed for linear operations and multiplication. This results in great inefficiency for common operations like bitwise AND and XOR. *Lookup arguments* are very helpful in these cases. All possible input-output pairs of a gate are put in a lookup table; to prove that the gates are evaluated honestly, the prover shows that the input-output pairs claimed can be found in the table. Popular lookup arguments include plookup [8] based on univariate polynomials, and Lasso [17] based on multilinear polynomials. Lasso exploits the fact that common operations are often *decomposable*; for example, a bitwise AND on 64-bit integers is the combination of four bitwise ANDs on 16-bit integers. This can dramatically reduce the sizes of the tables, provided they are of suitable structure. However, in both univariate and multivariate systems, lookup gates are still more costly than regular Plonk gates,

polygon	Naïve	Linear-only	Custom	Custom*
Constraints	817,505	497,156	432,730	429,166
Time	0.67s	2.33s	6.25s	169s
Memory	909MB	652MB	954MB	952MB

verify	Naïve	Linear-only	Custom	Custom*
Constraints	3,288,407	2,277,040	2,504,655	2,301,856
Time	2.63s	6.70s	27.6s	1200s
Memory	3.36GB	2.63GB	4.78GB	4.21GB

Table 5: Resource consumption of different converters with jellyfish turbo gates

Circuit	Constraints	Nonzero elements
pubkeygen	96,482 (+1,038)	469,496 (-71.6%)
eth_addr	248,418 (+1,038)	1,209,836 (-49.4%)
groupsig	251,985 (+1,047)	1,225,155 (-49.1%)
verify	1,524,000 (+15,096)	6,039,896 (-38.6%)

Table 6: Comparison of R1CS compiled by the original Circom and our modified Circom

and it is even more costly to introduce multiple lookup tables. Therefore, it might be necessary to carefully decide which gates/operations to implement as lookup, and which ones to implement in arithmetic circuits.

A.4. Plonk-based proof systems

Plonk [9] is a popular zero-knowledge succinct non-interactive argument of knowledge (SNARK) that has seen some application in blockchains. While successful, it is based on univariate polynomials and will inevitably involve FFTs. Its prover time is quasi-linear (due to the FFTs required) and does not scale well to large circuits.

Pianist [13] improves upon Plonk by distributing the proof generation process across multiple machines. With M sub-provers, it is able to prove M sub-circuits of size T each at a per-prover cost of $O(T \log T + M \log M)$, compared to $O(MT \log MT)$ (with a single prover) with plain Plonk. The communication costs are constant, and the proof sizes are the same as Plonk. However, Pianist retains the logarithmic factor from its underlying proof system Plonk, and hence is not optimal for large-scale application, where the FFT costs will become dominant.

HyperPlonk [7] adapts Plonk to the boolean hypercube with multilinear polynomials. Since HyperPlonk uses multivariate polynomials, it is able to forego FFTs and save the logarithmic factor. Its prover time is now linear, but the proof size and verifier time become logarithmic.

HyperPianist [12] is the distributed version of HyperPlonk, with several optimizations focused on prover costs. It is fully distributed like Pianist, while retaining the linear prover time of HyperPlonk. In addition, it can be equipped with Lasso lookup arguments.

A.5. Compilation of Plonk circuits

There are relatively few works in this field. Clap [18] is an automatic optimizer for Plonkish circuits that accepts an ‘eDSL’ written in Rust. Another work zkLLVM [15] aims to transpile LLVM IR into Plonk circuits. While the direction is promising, in the current state their tool is quite clumsy due to its tight integration with custom versions of LLVM and focus on their ‘proof market’ concept. It is also not clear if the involvement of LLVM IR would help or hinder the effective optimization of zero-knowledge circuits. In any case, we are also doubtful that either Clap or zkLLVM would be able to efficiently utilize high-degree gate equations and lookup arguments.

Circuit	Optimized Original	Greedy-BF Original	Optimized Modified	Greedy-BF Modified
pubkeygen	423,853	417,601	250,460	250,364
eth_addr	605,589	584,913	432,757	417,676
groupsig	610,837	589,829	437,426	422,610
verify	3,779,308	3,707,511	2,658,929	2,658,833

Table 7: Effectiveness of Plonkify with the original Circom versus the modified Circom: vanilla gates

Circuit	Linear-only Original	Custom* Original	Linear-only Modified	Custom* Modified
pubkeygen	206,125	208,362	148,526	150,509
eth_addr	373,038	366,483	315,409	308,528
groupsig	376,652	370,086	319,067	312,163
verify	2,277,040	2,301,856	1,908,554	1,931,934

Table 8: Effectiveness of Plonkify with the original Circom versus the modified Circom: jellyfish turbo gates

Circuit	R1CS	Vanilla Plonk	Jellyfish Plonk
polygon	229,847	855,296 ($3.7\times$)	429,166 ($1.9\times$)
pubkeygen	95,444	250,364 ($2.6\times$)	148,526 ($1.6\times$)
eth_addr	247,380	417,676 ($1.7\times$)	308,528 ($1.2\times$)
groupsig	250,938	422,610 ($1.7\times$)	312,163 ($1.2\times$)
verify	1,508,904	2,658,833 ($1.8\times$)	1,908,554 ($1.3\times$)

Table 9: Number of constraints: R1CS vs. Plonk